CE502

Salah Safi

Spring 2024/2025

Homework 1:

Total 10 points

Question 1 [4]: Suppose we have a CPU that is handling floating point numbers addition using 7 steps: Fetch operands, Compare exponents, Shift one operand, Add, Normalize result, Round result, Store result.

Suppose that Fetch and Store to L1 cache each take 2 nanoseconds and the remaining operations each take 1 nanoseconds.

Assume all operands are in the L1 cache:

1.1 How long does a floating point addition take with these assumptions?

1.2 How long will an **unpipelined** addition of 1000 pairs of floats take with these assumptions?

1.3 How long will a **pipelined** addition of 1000 pairs of floats take with these assumptions?

Assume the access to L2 cache is 10 nano-seconds, and the access to main memory takes 100 nano-seconds.

1.4 How long will a pipelined addition of 1000 pairs of floats take, if the first 250 pairs are in L1 cache, the second 250 pairs are in L2 cache and the remaining 500 pairs are in the main memory ?

Question 2 [4 points]: Assume we have 16-bit registers, an L1 cache that is 8-lines, and the line (cache block) size is 8 locations.

2.1 Assume an empty fully associative cache with FIFO replacement.

Consider the following sequence of load data from memory addresses, for each load operation, specify whether it is a cache hit or a cache miss (The address is in hexadecimal)?

Load data from **0x1000** location
Load data from **0x1004** location
Load data from **0x1008** location
Load data from **0x1009** location
Load data from **0x1110** location
Load data from **0xD000** location
Load data from **0x1001** location
Load data from **0x2101** location
Load data from **0x2201** location
Load data from **0x2301** location
Load data from **0x2401** location
Load data from **0x2501** location
Load data from **0x1005** location

2.2 Assume an empty direct mapped cache.

Consider the following sequence of loading data from memory addresses, for each load operation, specify whether it is a cache hit or a cache miss (The address is in hexadecimal) ?

Load data from **0x1000** location
Load data from **0x1004** location
Load data from **0x1008** location
Load data from **0x1009** location
Load data from **0x1110** location
Load data from **0xD000** location
Load data from **0x1001** location
Load data from **0x2101** location
Load data from **0x2201** location
Load data from **0x2301** location
Load data from **0x2401** location
Load data from **0x2501** location
Load data from **0x1005** location

Question 3 [2 points]:

In this exercise you will need to install and configure MPI**.** Message Passing Interface (MPI) is a standardized message-passing library interface specification.

Open MPI is an implementation for MPI:
https://www.open-mpi.org
Open MPI installation:
https://docs.open-mpi.org/en/v5.0.x/installing-open-mpi/quickstart.html

In this exercise I provided you with two C files, one for sequential processing and one for parallel processing.

We need to run the programs with different array sizes as in the tables below:

```
#define ARRAY_SIZE 100000000 // Size of the array
```

Commands to Compile:

gcc -o run_serial run_serial.c
mpicc -g -Wall -o run_parallel run_parallel.c

Commands to Run:
./run_serial
mpiexec -n 4 ./run_parallel

Please use the files provided and fill the following tables (**It is highly recommended to run each experiment 10 times and get the average time**) (you can stop at the number of cpus available on your machine)

Size	p	Tparallel (Avg.)
100M	2	
100M	3	
100M	4	
100M	5	
100M	6	
100M	7	
100M	8	

Size	Tserial (Avg)	p	Tparallel(Avg)	Speed-up	Efficiency
50M		2			
75M		3			
100M		4			
125M		5			
150M		6			